

Why This Problem Matters

Geometry problems show up less often than array or graph questions in interviews, but when they do, they separate candidates who pattern-match from candidates who actually reason about the shapes in front of them. LeetCode 1401, Circle and Rectangle Overlapping, looks like a coordinate-geometry exercise, but it's really testing something else: can you find the *right* thing to measure?

Most people's first instinct here is wrong, and it's wrong in a way that passes some test cases and fails others — which is exactly the kind of bug that survives code review and breaks in production. That makes this problem a great teaching example, not just for LeetCode, but for how collision detection actually works in games, UI hit-testing, and physics engines.

Problem Overview

You're given two shapes:

- A **circle**, defined by its center coordinates (x_center, y_center) and a **radius**.
- A **rectangle**, defined by its bottom-left corner $(x1, y1)$ and top-right corner $(x2, y2)$, where $x1 \leq x2$ and $y1 \leq y2$.

Your job is to return `True` if the circle and rectangle share at least one point, and `False` otherwise. Touching at a single point — the circle just grazing an edge or corner — still counts as overlapping.

Constraints keep the numbers small (radius and coordinates are bounded to a few thousand), so there's no need to worry about floating-point precision issues at scale, and the rectangle's corners are always given in the correct order, so you never have to normalize them.

Example

Let's use the case that trips up the naive approach:

- Circle: center $(0, 0)$, radius `1`
- Rectangle: bottom-left $(1, -1)$, top-right $(3, 1)$

The circle sits to the left of the rectangle. The rectangle's **y** range is $[-1, 1]$, and the circle's center **y** = `0` already falls inside that range. The rectangle's **x** range is $[1, 3]$, and the circle's center **x** = `0` falls outside it, to the left.

So the closest point on the rectangle to the circle's center is $(1, 0)$ — the x value snapped to the rectangle's left edge, the y value left alone. The distance from $(0, 0)$ to $(1, 0)$ is exactly 1 , which equals the radius. The circle touches the rectangle at exactly one point. Answer: `True`.

That single traced example is the whole problem in miniature — once you see how the closest point gets found, the general solution just repeats the same logic in both dimensions.

Intuition

Here's the mistake most people make first: measure the distance from the circle's center to the rectangle's center, and compare it to the radius.

Run that on the example above. The rectangle's center is $(2, 0)$. Distance from $(0, 0)$ to $(2, 0)$ is 2 , which is bigger than the radius of 1 . That says "no overlap." But we just showed the circle touches the rectangle's edge. The center-to-center distance is measuring the wrong thing entirely — the rectangle's center is almost never the point closest to the circle.

The real question isn't "how far apart are the centers?" It's "what's the single closest point on the rectangle to the circle's center?" That point could be:

- A **corner**, if the circle is diagonally outside the rectangle.
- A point on a **flat edge**, if the circle is directly above, below, left, or right of the rectangle.
- The circle's own center, if it's **already inside** the rectangle.

If that closest point lies within the circle's radius, the shapes overlap. If it doesn't, nothing on the rectangle is close enough, and they don't.

The elegant part is *how* you find that closest point without writing separate logic for corners, edges, and the inside case. You clamp the circle's x coordinate to the rectangle's $[x1, x2]$ range, and clamp its y coordinate to the rectangle's $[y1, y2]$ range:

```
closest_x = max(x1, min(x_center, x2))
closest_y = max(y1, min(y_center, y2))
```

If the center is already inside the rectangle, both `min` and `max` leave it unchanged — clamping is a no-op, and the "closest point" is the center itself, giving a distance of zero. If the center is outside on one axis, that axis gets pulled to the nearest edge. If it's outside on both axes, you land exactly on a corner. One formula, every case covered.

Brute Force Solution

Idea: Check the circle's distance to each of the rectangle's four corners and to each of its four edges as separate cases, then take the minimum.

Algorithm: 1. Compute distance from the circle's center to each corner. 2. For each edge, project the center onto the edge's line segment and compute that distance. 3. Also check whether the center is inside the rectangle. 4. Take the smallest of all these distances and compare it to the radius.

Advantages: Conceptually explicit — you can see exactly which case you're handling.

Disadvantages: It's four to eight separate distance calculations with different formulas for corners versus edges versus the interior case. It's easy to miss a case, and the code balloons for no real benefit, since a single clamp already produces the same closest point.

Time complexity: $O(1)$ — a fixed number of comparisons regardless of input. **Space complexity:** $O(1)$.

Note this "brute force" is still constant time — geometry problems like this rarely have a slower alternative, the difference is code complexity and correctness risk, not asymptotic speed.

Optimal Solution

The optimal approach is the clamp-and-compare method described in the intuition section. Here's the full breakdown:

Step	Operation	Purpose
1	<code>closest_x = max(x1, min(x_center, x2))</code>	Snap x into rectangle's horizontal bounds
2	<code>closest_y = max(y1, min(y_center, y2))</code>	Snap y into rectangle's vertical bounds
3	<code>dx = x_center - closest_x, dy = y_center - closest_y</code>	Distance components to closest point
4	<code>dx*dx + dy*dy <= radius*radius</code>	Compare squared distance to squared radius

Step 4 avoids a square root. Since both sides of the comparison are non-negative, squaring preserves the ordering — `a <= b` if and only if `a2 <= b2` for non-negative values — so skipping `sqrt()` costs nothing in correctness and saves a slower floating-point operation.

Python Code

```
def check_overlap(radius: int, x_center: int, y_center: int,
                  x1: int, y1: int, x2: int, y2: int) -> bool:
```

```
# Find the point on the rectangle closest to the circle's center
closest_x = max(x1, min(x_center, x2))
closest_y = max(y1, min(y_center, y2))

# Compare squared distance to squared radius to avoid sqrt
dx = x_center - closest_x
dy = y_center - closest_y
return dx * dx + dy * dy <= radius * radius
```

Code Walkthrough

- `min(x_center, x2)` pulls `x_center` down if it's past the rectangle's right edge; otherwise it stays as `x_center`.
- `max(x1, ...)` then pushes that value up if it's past the left edge; otherwise it's unchanged.
- The same two-step clamp happens on the `y` axis against the top and bottom edges.
- `dx` and `dy` are the horizontal and vertical distance from the circle's center to that clamped point.
- `dx * dx + dy * dy` is the squared Euclidean distance — no `sqrt` needed.
- The final comparison against `radius * radius` answers the overlap question directly.

Dry Run

Using circle `(0, 0)`, radius `1`, rectangle `(1, -1)` to `(3, 1)`:

1. `closest_x = max(1, min(0, 3)) = max(1, 0) = 1`
2. `closest_y = max(-1, min(0, 1)) = max(-1, 0) = 0`
3. `dx = 0 - 1 = -1`, `dy = 0 - 0 = 0`
4. `dx*dx + dy*dy = 1 + 0 = 1`
5. `radius*radius = 1`
6. `1 <= 1` → `True`

The circle touches the rectangle's edge exactly at the boundary, and the function correctly returns `True`.

Complexity Analysis

Time: $O(1)$. The solution performs a fixed number of arithmetic operations — two `min` / `max` pairs, two subtractions, two multiplications, one addition, one comparison — regardless of the magnitude of the inputs. There are no loops, no recursion, and no data structure traversal.

Space: $O(1)$. Only a handful of scalar variables are used; nothing scales with input size.

These bounds are tight because the problem is fundamentally a fixed-size geometric computation — there's no way to do less work than compute the closest point and check one distance.

Alternative Solutions

You could check the four corners and four edges as separate cases (the brute force approach above), but it produces the same result with more code and more chances for an off-by-one or missed case. There's no algorithmically different approach that improves on $O(1)$ — this problem is already at its theoretical floor. The only real choice is between the unified clamp formula and the verbose case-by-case version.

Edge Cases

- **Circle's center exactly inside the rectangle:** Clamping is a no-op, distance is `0`, which is always `<= radius2`, so it returns `True`.
- **Circle far from the rectangle:** Clamped point is a corner, squared distance exceeds `radius2`, returns `False`.
- **Circle exactly touching an edge or corner:** Distance equals radius exactly — the `<=` comparison must include equality, or this case incorrectly returns `False`.
- **Zero radius:** The circle degenerates to a point; the same clamp logic still works, checking if that point lies within (or on) the rectangle.
- **Rectangle is a line or point** (`x1 == x2` or `y1 == y2`): Clamping still works — one axis has a single valid value, and the comparison still holds.

Common Mistakes

1. **Using center-to-center distance** instead of finding the closest point — fails whenever the circle overlaps a flat edge rather than sitting near the rectangle's center.
2. **Using strict `<` instead of `<=`** — misses the case where the circle exactly touches the rectangle, which the problem explicitly counts as overlap.
3. **Taking the square root unnecessarily** — not wrong, just slower; comparing squared values is equivalent and cheaper.
4. **Writing separate branches for corners, edges, and interior** — error-prone and redundant when a single clamp handles all three.

5. **Assuming rectangle corners might be unordered** — the problem guarantees $x1 \leq x2$ and $y1 \leq y2$, so adding a normalization step is wasted code.

Interview Questions

- How would you extend this to check overlap between two circles, or two rectangles?
- What changes if the rectangle can be rotated (not axis-aligned)?
- Can you compute the actual overlapping area, not just whether it overlaps?
- How does this generalize to 3D — a sphere and a box?
- Why is comparing squared distances valid here, and when would it *not* be valid?

Similar Problems

- **LeetCode 223 (Rectangle Area)**: Also axis-aligned rectangle geometry, focused on computing overlapping area instead of a boolean check.
- **LeetCode 1266 (Minimum Time Visiting All Points)**: Builds distance intuition using Chebyshev distance rather than Euclidean.
- **LeetCode 836 (Rectangle Overlap)**: The pure rectangle-rectangle version of this same "closest point / bounds check" family.
- **LeetCode 1232 (Check If It Is a Straight Line)**: Different shape, same theme — reasoning about coordinate geometry directly rather than brute-forcing points.

Key Takeaways

Overlap and collision problems usually reduce to one question: what's the closest point between the two shapes? Once you find that, a single distance check settles everything. Clamping is the tool that finds the closest point on an axis-aligned rectangle in one line per axis, covering corners, edges, and the inside case simultaneously. And comparing squared distances instead of taking a square root is a small, free optimization worth having as a reflex.

Watch the Video

If you want to see this traced out visually, step by step, with the quiz at the end to test your understanding, watch the full walkthrough here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series that works through LeetCode problems, real-world engineering concepts, and interview prep the way an experienced engineer actually thinks through them — building intuition first, writing code second, and never skipping the "why."

Call To Action

If this helped make LeetCode 1401 click, give the video a like on YouTube and subscribe for more daily problem breakdowns. For deeper dives and behind-the-scenes notes on the series, subscribe to the Substack. Drop a comment with which problem you want covered next, and share this with anyone prepping for coding interviews.

Quick Review

Circle-rectangle overlap: what's the trick instead of checking all edges?

Clamp circle center to rectangle bounds to find closest point, then compare distance to radius.

Time complexity of closest-point clamping approach?

$O(1)$ — just min/max clamps and one distance comparison, no loops.

Space complexity of the clamp-and-check solution?

$O(1)$ — a few scalar variables (clamped x, y), no extra structures.

Common bug when comparing distance to radius?

Forgetting to compare squared distance to radius squared, or taking sqrt unnecessarily and hitting float precision issues.

Clamping-to-closest-point vs brute-force edge/corner checks?

Clamping handles all cases (inside, edge, corner) in one formula; brute-force needs separate cases and is error-prone.

Interview follow-up: circle-circle overlap instead of circle-rectangle?

Compare distance between centers to sum of radii — no clamping needed since both shapes are round.

Circle and Rectangle Overlapping — free Python cheat sheet